

Staff Detection and Video Tracking

AI Evaluation | Solution documentation | Updated 11 September 2026

1. Objective and model

Identify frames containing staff and locate their XY coordinates in overhead video. The staff name tag defines the Staff class. This solution uses the project's trained YOLO26n best.pt checkpoint with the existing ByteTrack configuration. The model predicts whole-person Staff boxes from learned examples; it does not separately detect or read the tag. Tracking associates detections over time and assigns IDs.

2. Original labels and training

The supplied sample.mp4 has 1,341 frames at 25 FPS and 960 x 720 pixels. dataset_create.py extracts every fifth frame, producing 269 images. parse_ndjson.py converts original split NDJSON annotations to YOLO labels. The dataset has 215 training images with 48 Staff boxes and 54 validation images with 11 Staff boxes. Other images have empty labels according to the original annotations.

Use the existing dataset/train and dataset/val labels. Image frame_NNNN.jpg maps to one-based video frame $5 \times \text{NNNN} + 1$; image 0063 therefore maps to frame 316. Intermediate frames are unlabelled, not automatically staff-absent. Class-and-box labels do not contain person identities, so they cannot establish ID-switch accuracy.

The saved model was trained for 50 epochs, at image size 640 and batch size 4, with GPU 0 and two workers. The current optional training script is train.py; yolo26n.yaml specifies dataset paths and the Staff class. Despite its filename, this is a dataset configuration, not a model architecture. Tracking loads existing best.pt weights and requires no retraining.

3. Tracking pipeline and outputs

tracking.py decodes every frame and calls model.track with persist=True, confidence 0.1, image size 640 and class 0. custom_tracker.yaml selects ByteTrack with high/new-track thresholds 0.6, low threshold 0.1, matching threshold 0.6, buffer 60 and score fusion enabled. A callback captures postprocessed detector boxes before tracker mutation in the same inference pass.

INPUT	DETECTOR	TRACKER	OUTPUT
sample.mp4	Trained best.pt	ByteTrack	MP4 + JSON records

frames.jsonl records every decoded frame, including frames without tracks: frame number, timestamp, detections, track IDs, confidence, bounding boxes, centre XY and diagnostic status. staff_present means at least one returned track, not verified ground truth. A missing detection only indicates failure if staff should be present.

XY is the floating-point box centre: $((x1+x2)/2, (y1+y2)/2)$, in pixels from the top-left, x rightward and y downward. These are not calibrated floor coordinates. --save-video writes annotated.mp4 with grey detector boxes and coloured tracks, IDs and trails. Trails restart after gaps. --save-images optionally writes JPEGs. Visual output is disabled unless requested.

Evidence: current scripts/YAML; original train/val labels; saved results.csv (epoch 50); full-run frames.jsonl and summary.json from 10 September 2026. See README for full commands. Training/export scripts retain local paths; exported-model parity is untested.

Results and Reproduction

4. Recorded results

Final-epoch box-detection metrics from the saved training results.csv are below. They are not tracking accuracy or a separate evaluation of best.pt.

Precision	Recall	mAP50	mAP50-95
99.41%	90.91%	90.50%	47.10%

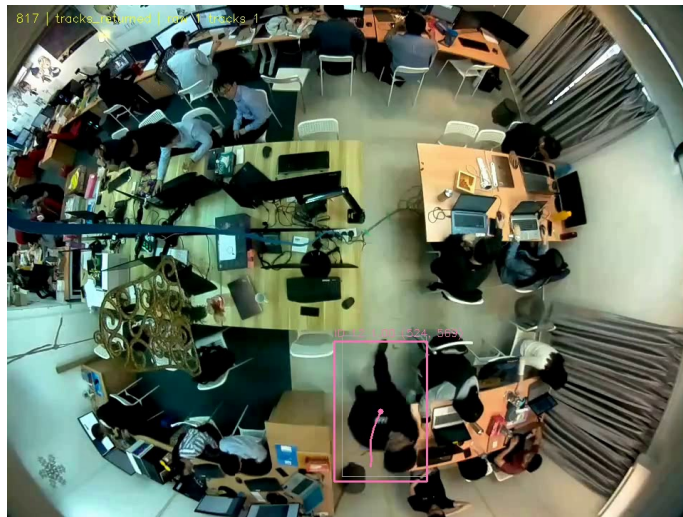


Figure 1. Recorded tracking output, frame 817: predicted Staff track ID 12 and centre approximately (524, 569). Trails show observed motion; confidence is not an accuracy measurement.

The recorded full run processed all 1,341 frames: 208 had returned tracks, 99 had detections without tracks, and 1,034 had no detections. These are diagnostic counts, not accuracy metrics. It measured about 64.5 tracking calls/sec and 43.3 pipeline frames/sec with video saving, using GPU 0 and Ultralytics 8.4.144. Performance depends on hardware and settings.

Tracking-call FPS excludes the first call and includes detector capture and GPU synchronization. Pipeline FPS includes model setup, warm-up, decoding, logging, enabled drawing/writing and resource release; summary serialization is excluded. summary.json records settings, source metadata, completeness and timing.

5. Running the model

From the repository root, activate the Python environment and install requirements.txt if needed. Put the source at input_video/sample.mp4 and run `python tracking.py --save-video`. The bundled checkpoint is best.pt in the saved run folder ending 04-32-58_e50_b4. Tracking selects GPU 0 when available, otherwise CPU.

For another video: `python tracking.py --source "D:\Videos\test.mp4" --save-video --device cpu`. Each run creates runs/tracking_<timestamp>. --output selects an unused directory; existing directories are rejected. The MP4 uses mp4v, whose browser support varies. Local source video/data and generated run outputs are excluded from GitHub.

6. Limitations

The single-video split has temporal similarity: 52 of 54 validation images have an adjacent sampled training image. Results may be optimistic; test unseen video before claiming generalization. Hidden tags do not automatically make identifiable staff absent. The model may learn clothing/background cues; tracking may miss frames or change IDs. There is no offline backfill or explicit badge verification.